# FIFA 24 Player Stats Analysis

Authors: Erik Argueta & Logan Brenner

In [2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from plotnine import *
```

# Get The Data

In [3]:
```
fifa_df = pd.read_csv("Material/fifa_stats.csv")
```

In [4]:
```
fifa_df.head(10)
```

Out[4]:

| | ID | country | height | weight | age | club | ball_control | dribbling | mar |
|---|---|---|---|---|---|---|---|---|---|
| **0** | PS1 | Egypt | 175 | 71 | 31 | Liverpool | 86 | 88 | |
| **1** | PS2 | Germany | 177 | 75 | 28 | FC Bayern München | 85 | 82 | |
| **2** | PS3 | Germany | 183 | 80 | 29 | 1. FSV Mainz 05 | 73 | 69 | |
| **3** | PS4 | France | 196 | 74 | 23 | Leeds United | 35 | 20 | |
| **4** | PS5 | Argentina | 176 | 78 | 35 | Al Shabab | 82 | 79 | |
| **5** | PS6 | Spain | 194 | 82 | 28 | Rayo Vallecano | 66 | 35 | |
| **6** | PS7 | Spain | 177 | 73 | 29 | RCD Espanyol | 73 | 68 | |
| **7** | PS8 | Norway | 170 | 70 | 30 | Adana Demirspor | 74 | 73 | |
| **8** | PS9 | Colombia | 176 | 73 | 24 | Columbus Crew | 73 | 77 | |
| **9** | PS10 | Brazil | 179 | 77 | 31 | Atlético Mineiro | 73 | 56 | |

10 rows × 41 columns

In [5]:
```
fifa_df.shape
```

Out[5]:
```
(5682, 41)
```

# Exploring the Data (Exploratory Data Analysis - EDA)

In [6]:
```
fifa_df.dtypes
```

Out[6]:
```
ID                  str
country             str
height            int64
weight            int64
age               int64
club                str
ball_control      int64
dribbling         int64
marking         float64
slide_tackle      int64
stand_tackle      int64
aggression        int64
reactions         int64
att_position      int64
interceptions     int64
vision            int64
composure         int64
crossing          int64
short_pass        int64
long_pass         int64
acceleration      int64
stamina           int64
strength          int64
balance           int64
sprint_speed      int64
agility           int64
jumping           int64
heading           int64
shot_power        int64
finishing         int64
long_shots        int64
curve             int64
fk_acc            int64
penalties         int64
volleys           int64
gk_positioning    int64
gk_diving         int64
gk_handling       int64
gk_kicking        int64
gk_reflexes       int64
value           float64
dtype: object
```

In [7]:
```
(
    fifa_df
    .duplicated().any()
)
```

Out[7]: np.False_

In [8]:
```
fifa_df.describe().T
```

Out[8]:

| | count | mean | std | min | 25% | 50% | |
|---|---|---|---|---|---|---|---|
| **height** | 5682.0 | 1.816705e+02 | 6.829238e+00 | 156.0 | 177.0 | 182.0 | 1 |
| **weight** | 5682.0 | 7.528229e+01 | 6.998971e+00 | 54.0 | 70.0 | 75.0 | |
| **age** | 5682.0 | 2.631626e+01 | 4.729967e+00 | 17.0 | 23.0 | 26.0 | |
| **ball_control** | 5682.0 | 5.891218e+01 | 1.656707e+01 | 8.0 | 55.0 | 63.0 | |
| **dribbling** | 5682.0 | 5.612848e+01 | 1.877208e+01 | 5.0 | 51.0 | 62.0 | |
| **marking** | 0.0 | NaN | NaN | NaN | NaN | NaN | |
| **slide_tackle** | 5682.0 | 4.672844e+01 | 2.051943e+01 | 7.0 | 27.0 | 54.0 | |
| **stand_tackle** | 5682.0 | 4.882084e+01 | 2.097597e+01 | 7.0 | 30.0 | 57.0 | |
| **aggression** | 5682.0 | 5.632401e+01 | 1.684641e+01 | 11.0 | 45.0 | 60.0 | |
| **reactions** | 5682.0 | 6.195935e+01 | 8.893309e+00 | 32.0 | 56.0 | 62.0 | |
| **att_position** | 5682.0 | 5.072492e+01 | 1.978064e+01 | 3.0 | 40.0 | 56.0 | |
| **interceptions** | 5682.0 | 4.738912e+01 | 2.045133e+01 | 3.0 | 28.0 | 54.0 | |
| **vision** | 5682.0 | 5.446568e+01 | 1.370529e+01 | 10.0 | 45.0 | 56.0 | |
| **composure** | 5682.0 | 5.862091e+01 | 1.202410e+01 | 13.0 | 52.0 | 60.0 | |
| **crossing** | 5682.0 | 4.979039e+01 | 1.789805e+01 | 6.0 | 39.0 | 54.0 | |
| **short_pass** | 5682.0 | 5.933333e+01 | 1.432602e+01 | 11.0 | 55.0 | 63.0 | |
| **long_pass** | 5682.0 | 5.391077e+01 | 1.460136e+01 | 9.0 | 46.0 | 57.0 | |
| **acceleration** | 5682.0 | 6.475466e+01 | 1.529860e+01 | 12.0 | 57.0 | 68.0 | |
| **stamina** | 5682.0 | 6.337733e+01 | 1.610590e+01 | 14.0 | 56.0 | 66.0 | |
| **strength** | 5682.0 | 6.538120e+01 | 1.262405e+01 | 25.0 | 58.0 | 66.5 | |
| **balance** | 5682.0 | 6.407040e+01 | 1.449544e+01 | 20.0 | 56.0 | 66.0 | |
| **sprint_speed** | 5682.0 | 6.495899e+01 | 1.511404e+01 | 13.0 | 57.0 | 68.0 | |
| **agility** | 5682.0 | 6.378054e+01 | 1.485787e+01 | 21.0 | 56.0 | 66.0 | |
| **jumping** | 5682.0 | 6.508501e+01 | 1.227452e+01 | 23.0 | 57.0 | 66.0 | |
| **heading** | 5682.0 | 5.224534e+01 | 1.735946e+01 | 6.0 | 45.0 | 55.0 | |
| **shot_power** | 5682.0 | 5.818268e+01 | 1.297192e+01 | 20.0 | 49.0 | 59.0 | |
| **finishing** | 5682.0 | 4.635428e+01 | 1.982264e+01 | 4.0 | 31.0 | 51.0 | |
| **long_shots** | 5682.0 | 4.716420e+01 | 1.946337e+01 | 5.0 | 33.0 | 51.0 | |
| **curve** | 5682.0 | 4.809979e+01 | 1.808667e+01 | 7.0 | 36.0 | 50.0 | |
| **fk_acc** | 5682.0 | 4.329954e+01 | 1.708647e+01 | 4.0 | 32.0 | 43.0 | |
| **penalties** | 5682.0 | 4.817494e+01 | 1.578174e+01 | 8.0 | 39.0 | 49.0 | |
| **volleys** | 5682.0 | 4.293189e+01 | 1.791262e+01 | 4.0 | 30.0 | 44.0 | |
| **gk_positioning** | 5682.0 | 1.618761e+01 | 1.714657e+01 | 2.0 | 8.0 | 11.0 | |

| | count | mean | std | min | 25% | 50% | |
|---|---|---|---|---|---|---|---|
| **gk_diving** | 5682.0 | 1.643805e+01 | 1.763526e+01 | 2.0 | 8.0 | 11.0 | |
| **gk_handling** | 5682.0 | 1.619183e+01 | 1.699315e+01 | 2.0 | 8.0 | 11.0 | |
| **gk_kicking** | 5682.0 | 1.607198e+01 | 1.680867e+01 | 2.0 | 8.0 | 11.0 | |
| **gk_reflexes** | 5682.0 | 1.657022e+01 | 1.794215e+01 | 3.0 | 8.0 | 11.0 | |
| **value** | 4546.0 | 2.231722e+06 | 7.575116e+06 | 4.0 | 350.0 | 775.0 | 16000 |

# Categorizing Variables

In [9]:
```
fifa_cleaning = (
    fifa_df
)
```

## Remove Unnecessary Columns

In [10]:
```
fifa_cleaning.drop(columns=['gk_positioning', 'marking', 'gk_diving', 'gk_ha
```

In [11]:
```
fifa_cleaning.shape
```

Out[11]:
```
(5682, 35)
```

## Exploring Category: club

In [12]:
```
fifa_cleaning["club"].value_counts()
```

Out[12]:
```
club
River Plate          21
Liverpool            17
Lanús                17
Arsenal              16
Girona FC            16
                     ..
Hungary               1
Milan                 1
Sweden                1
Argentina             1
FC Bayern M�nchen     1
Name: count, Length: 689, dtype: int64
```

In [13]:
```
fifa_cleaning['club'] = fifa_cleaning['club'].str.strip()
```

In [14]:
```
(
    fifa_cleaning
    .query('`club` == "River Plate"')
)
```

Out[14]:

| | ID | country | height | weight | age | club | ball_control | dribbling | sli |
|---|---|---|---|---|---|---|---|---|---|
| **73** | PS74 | Chile | 178 | 76 | 29 | River Plate | 70 | 71 | |
| **343** | PS344 | Argentina | 182 | 77 | 30 | River Plate | 74 | 68 | |
| **594** | PS595 | Argentina | 178 | 75 | 24 | River Plate | 68 | 76 | |
| **596** | PS597 | Argentina | 185 | 89 | 31 | River Plate | 70 | 65 | |
| **806** | PS807 | Argentina | 182 | 67 | 33 | River Plate | 81 | 80 | |
| **825** | PS826 | Uruguay | 167 | 65 | 26 | River Plate | 78 | 82 | |
| **1095** | PS1096 | Uruguay | 169 | 65 | 26 | River Plate | 76 | 76 | |
| **1412** | PS1413 | Argentina | 184 | 79 | 26 | River Plate | 21 | 24 | |
| **2015** | PS2016 | Uruguay | 178 | 70 | 24 | River Plate | 53 | 51 | |
| **2377** | PS2378 | Argentina | 187 | 85 | 25 | River Plate | 19 | 16 | |
| **2739** | PS2740 | Uruguay | 175 | 71 | 18 | River Plate | 63 | 64 | |
| **2918** | PS2919 | Uruguay | 178 | 80 | 38 | River Plate | 81 | 74 | |
| **3112** | PS3113 | Uruguay | 191 | 78 | 21 | River Plate | 56 | 52 | |
| **3520** | PS3521 | Uruguay | 173 | 73 | 39 | River Plate | 62 | 68 | |
| **3898** | PS3899 | Uruguay | 188 | 80 | 20 | River Plate | 49 | 49 | |
| **3907** | PS3908 | Uruguay | 187 | 82 | 20 | River Plate | 19 | 9 | |
| **4614** | PS4615 | Paraguay | 182 | 76 | 25 | River Plate | 67 | 64 | |
| **4624** | PS4625 | Argentina | 170 | 69 | 35 | River Plate | 82 | 79 | |
| **4876** | PS4877 | Argentina | 179 | 76 | 30 | River Plate | 64 | 64 | |
| **5426** | PS5427 | Uruguay | 180 | 76 | 36 | River Plate | 38 | 33 | |
| **5456** | PS5457 | Uruguay | 181 | 77 | 31 | River Plate | 52 | 40 | |

21 rows × 35 columns

# Normalizing value

In [15]:
```
fifa_cleaning["log_market_value"] = np.log(fifa_df['value'])
```

In [16]:
```
(
    fifa_cleaning
    .filter(['log_market_value'])
    .sort_values(by='log_market_value', ascending=False)
)
```

Out[16]:

| | log_market_value |
|---|---|
| **823** | 18.849211 |
| **822** | 18.849211 |
| **852** | 18.627695 |
| **881** | 18.493001 |
| **882** | 18.430631 |
| **...** | ... |
| **5677** | NaN |
| **5678** | NaN |
| **5679** | NaN |
| **5680** | NaN |
| **5681** | NaN |

5682 rows × 1 columns

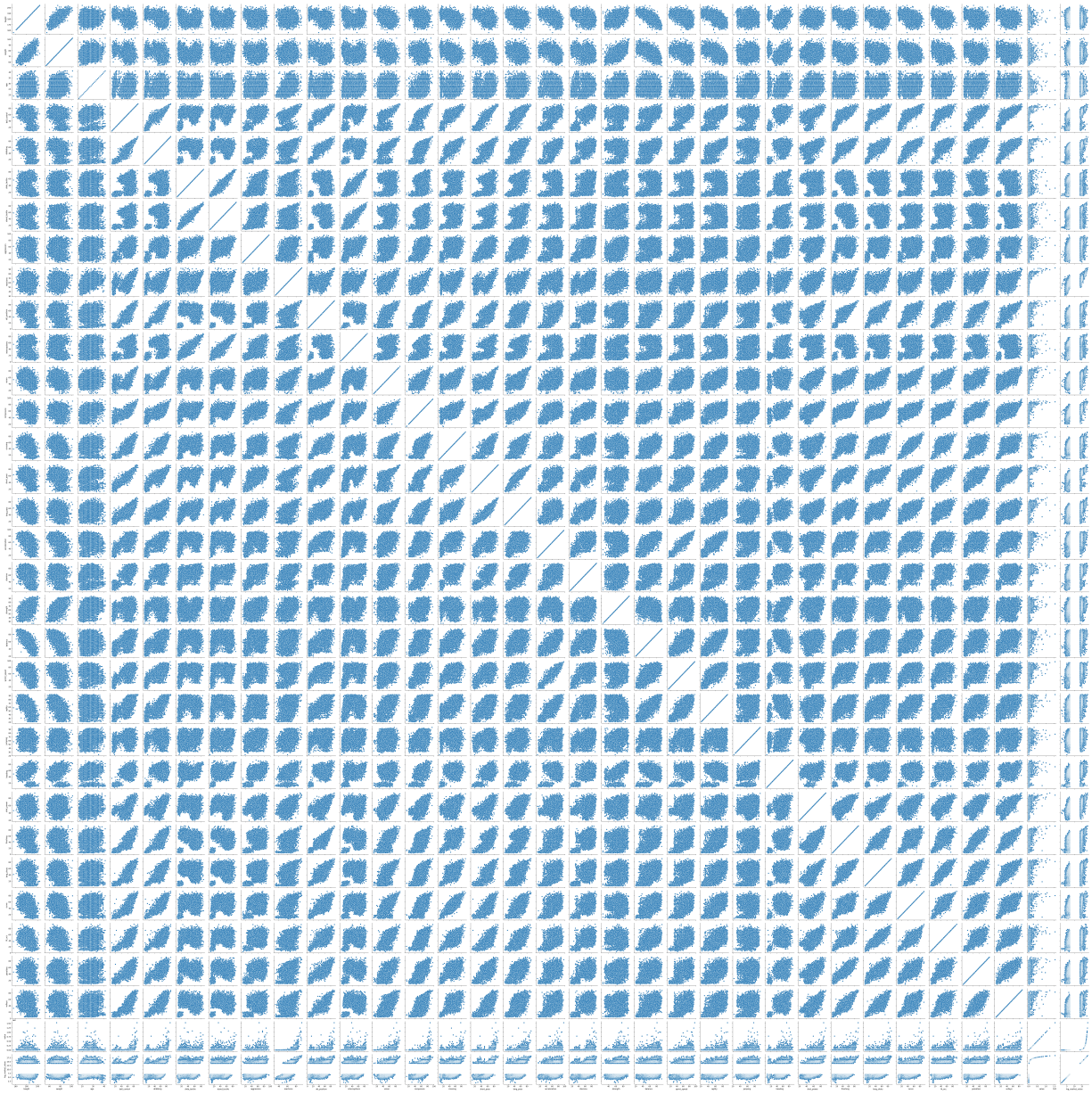
In [17]:
```
fifa_cleaning = (
    fifa_df.astype({'country': 'category', 'club': 'category'})
)
```

# Data Visualization

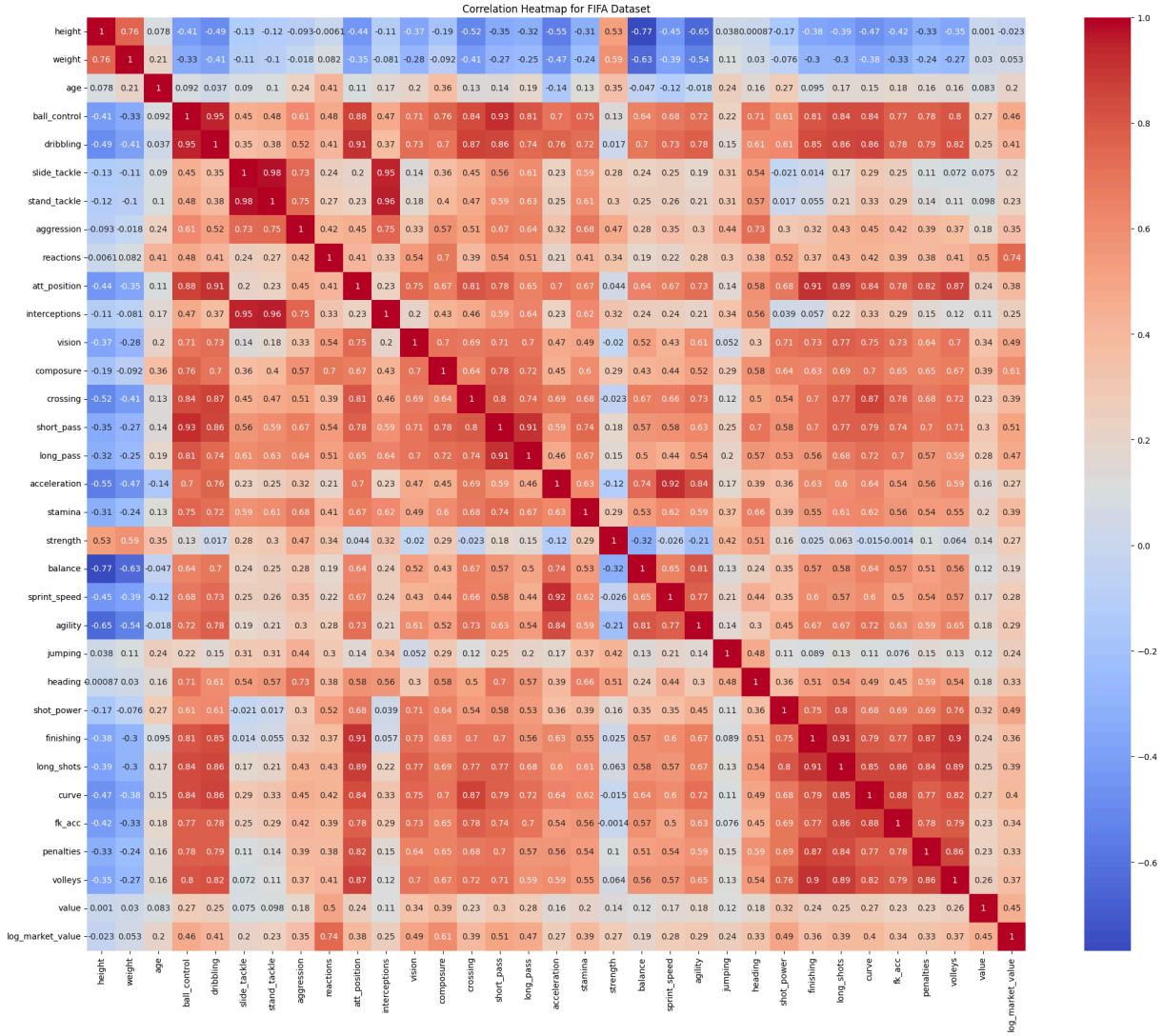
In [18]:
```
fifa_cleaning.dtypes
```

Out[18]:
```
ID                    str
country            category
height                int64
weight                int64
age                   int64
club               category
ball_control          int64
dribbling             int64
slide_tackle          int64
stand_tackle          int64
aggression            int64
reactions             int64
att_position          int64
interceptions         int64
vision                int64
composure             int64
crossing              int64
short_pass            int64
long_pass             int64
acceleration          int64
stamina               int64
strength              int64
balance               int64
sprint_speed          int64
agility               int64
jumping               int64
heading               int64
shot_power            int64
finishing             int64
long_shots            int64
curve                 int64
fk_acc                int64
penalties             int64
volleys               int64
value               float64
log_market_value    float64
dtype: object
```

In [19]:
```
g = sns.PairGrid(fifa_cleaning)
g.map(sns.scatterplot)
plt.show()
```



In [20]:
```
corr_matrix = fifa_cleaning.select_dtypes('number').corr()

plt.figure(figsize=(25,20))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap for FIFA Dataset")
plt.show()
```




# Strongest Correlations:

log_market_value x ['reactions', 'composure', 'short_pass']

In [21]:
```
(
    fifa_cleaning
    .filter(['ID','club', 'reactions', 'composure', 'short_pass', 'value', '
)
```

Out[21]:

| | ID | club | reactions | composure | short_pass | value | log_ |
|---|---|---|---|---|---|---|---|
| **0** | PS1 | Liverpool | 92 | 90 | 84 | 99500000.0 | |
| **1** | PS2 | FC Bayern München | 88 | 85 | 87 | 97000000.0 | |
| **2** | PS3 | 1. FSV Mainz 05 | 77 | 76 | 74 | 9500000.0 | |
| **3** | PS4 | Leeds United | 74 | 67 | 50 | 9500000.0 | |
| **4** | PS5 | Al Shabab | 79 | 80 | 83 | 9500000.0 | |
| **...** | ... | ... | ... | ... | ... | ... | |
| **5677** | PS5678 | Wilstermann | 65 | 67 | 23 | NaN | |
| **5678** | PS5679 | Venezia | 66 | 39 | 31 | NaN | |
| **5679** | PS5680 | Barrow | 55 | 60 | 27 | NaN | |
| **5680** | PS5681 | América Mineiro | 58 | 29 | 26 | NaN | |
| **5681** | PS5682 | Brøndby IF | 53 | 45 | 48 | NaN | |

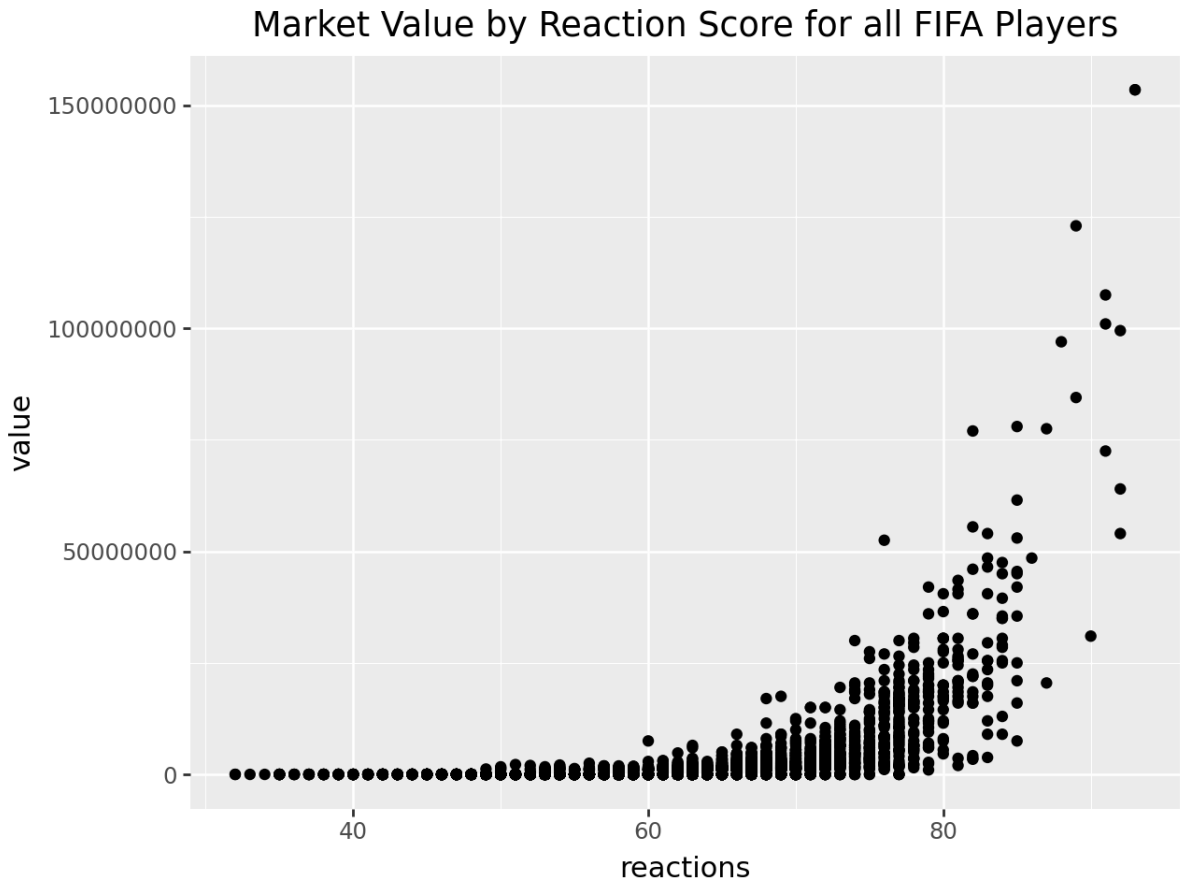
5682 rows × 7 columns

In [22]:

```
(
    ggplot(fifa_cleaning)
    + aes(x="reactions", y="value")
    + geom_point()
    + labs(
        title="Market Value by Reaction Score for all FIFA Players"
    )
)
```

```
/home/neuro/Documents/github/fifa-ml/.venv/lib/python3.12/site-packages/plot
nine/layer.py:374: PlotnineWarning: geom_point : Removed 1136 rows containin
g missing values.
```

Out[22]:

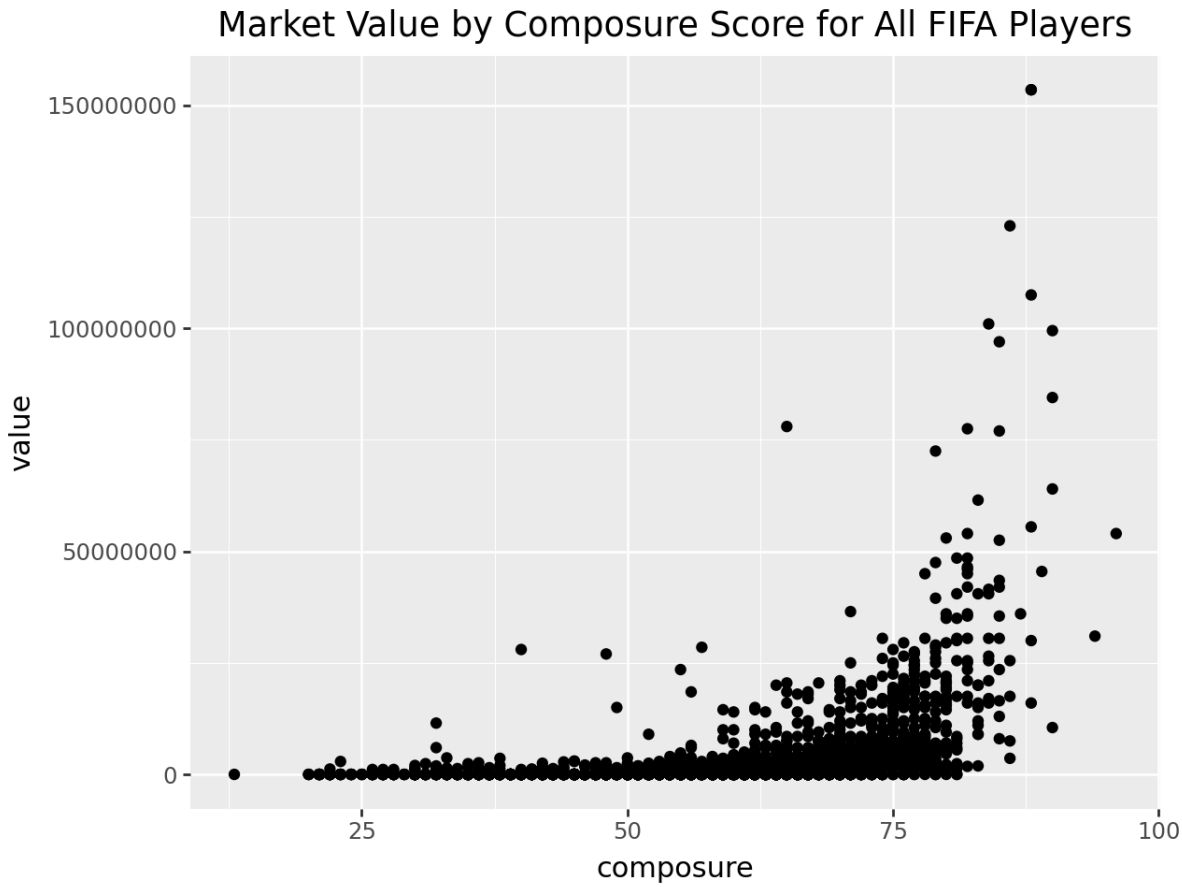



In [23]:

```
(
    ggplot(fifa_cleaning)
    + aes(x="composure", y="value")
    + geom_point()
    + labs(
        title="Market Value by Composure Score for All FIFA Players"
    )
)
```

```
/home/neuro/Documents/github/fifa-ml/.venv/lib/python3.12/site-packages/plot
nine/layer.py:374: PlotnineWarning: geom_point : Removed 1136 rows containin
g missing values.
```

Out[23]:

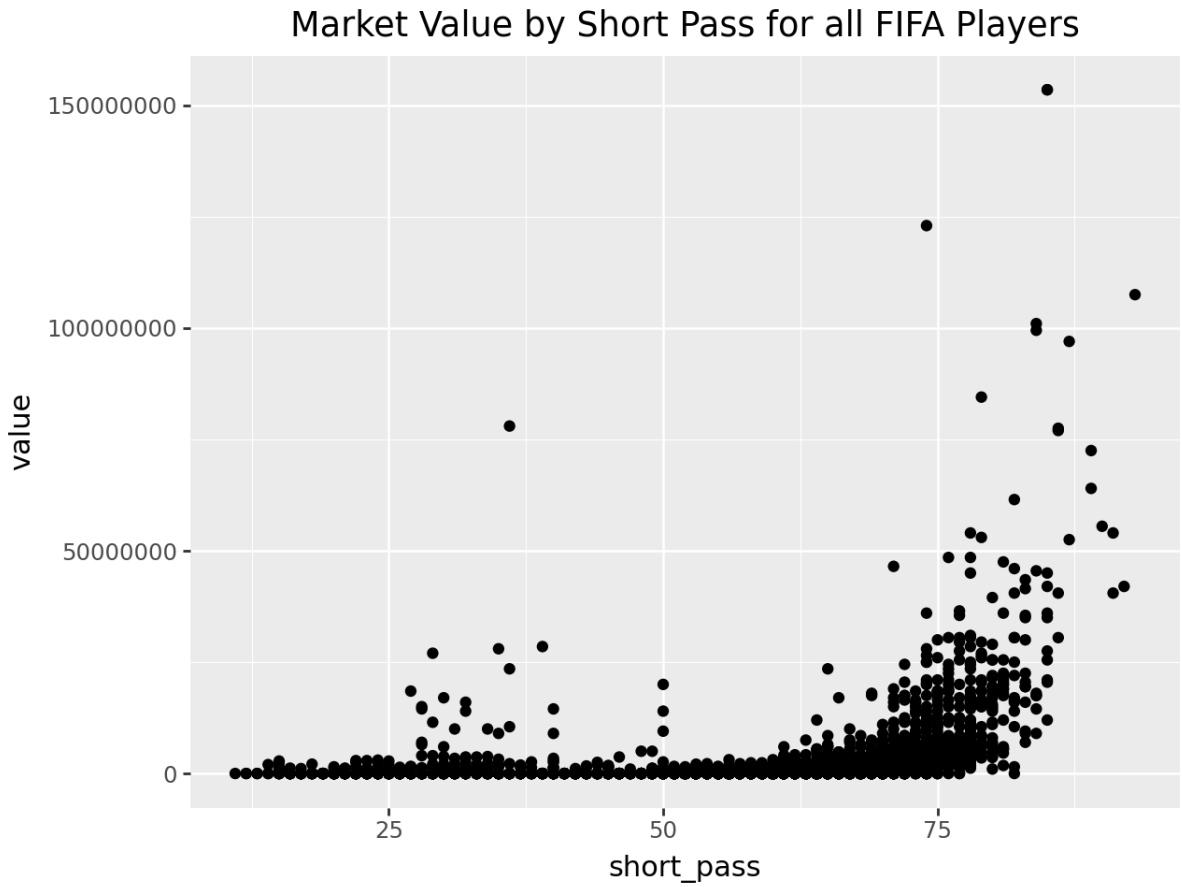



In [24]:

```
(
    ggplot(fifa_cleaning)
    + aes(x="short_pass", y="value")
    + geom_point()
    + labs(
        title="Market Value by Short Pass for all FIFA Players"
    )
)
```

```
/home/neuro/Documents/github/fifa-ml/.venv/lib/python3.12/site-packages/plot
nine/layer.py:374: PlotnineWarning: geom_point : Removed 1136 rows containin
g missing values.
```

Out[24]:




# Final Remarks

Correlation of variables in predicting `value` or `log_market_value`

- reactions (0.74)
- composure (0.61)
- short_pass (0.51)

**NOTE:** There are 1136 NaN `value` and `log_market_value`. Not sure what to do with them...

# Preparing the Data for Linear Regression

In [24]:
```
#Drop rows where there is no value
import numpy as np

df_model = fifa_cleaning.dropna(subset=['value']).copy()

df_model['log_market_value'] = np.log1p(df_model['value'])

df_model[['value', 'log_market_value']].head()
```

Out[24]:

| | value | log_market_value |
|---|---|---|
| **0** | 99500000.0 | 18.415668 |
| **1** | 97000000.0 | 18.390222 |
| **2** | 9500000.0 | 16.066802 |
| **3** | 9500000.0 | 16.066802 |
| **4** | 9500000.0 | 16.066802 |

In [25]:
```
#Some values were really low like like 1500 Id assume it should be 1.5 mil s
df_model['value_corrected'] = np.where(
    df_model['value'] < 10000,
    df_model['value'] * 1000,
    df_model['value']
)

df_model['log_market_value_corrected'] = np.log1p(df_model['value_corrected'
```

In [26]:
```
features = [
    'reactions',
    'composure',
    'short_pass',
    'ball_control',
    'vision',
    'long_pass',
    'shot_power',
    'finishing',
    'age'
]

X = df_model[features]
y = df_model['log_market_value_corrected']

X = X.fillna(X.median())
```

In [27]:
```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

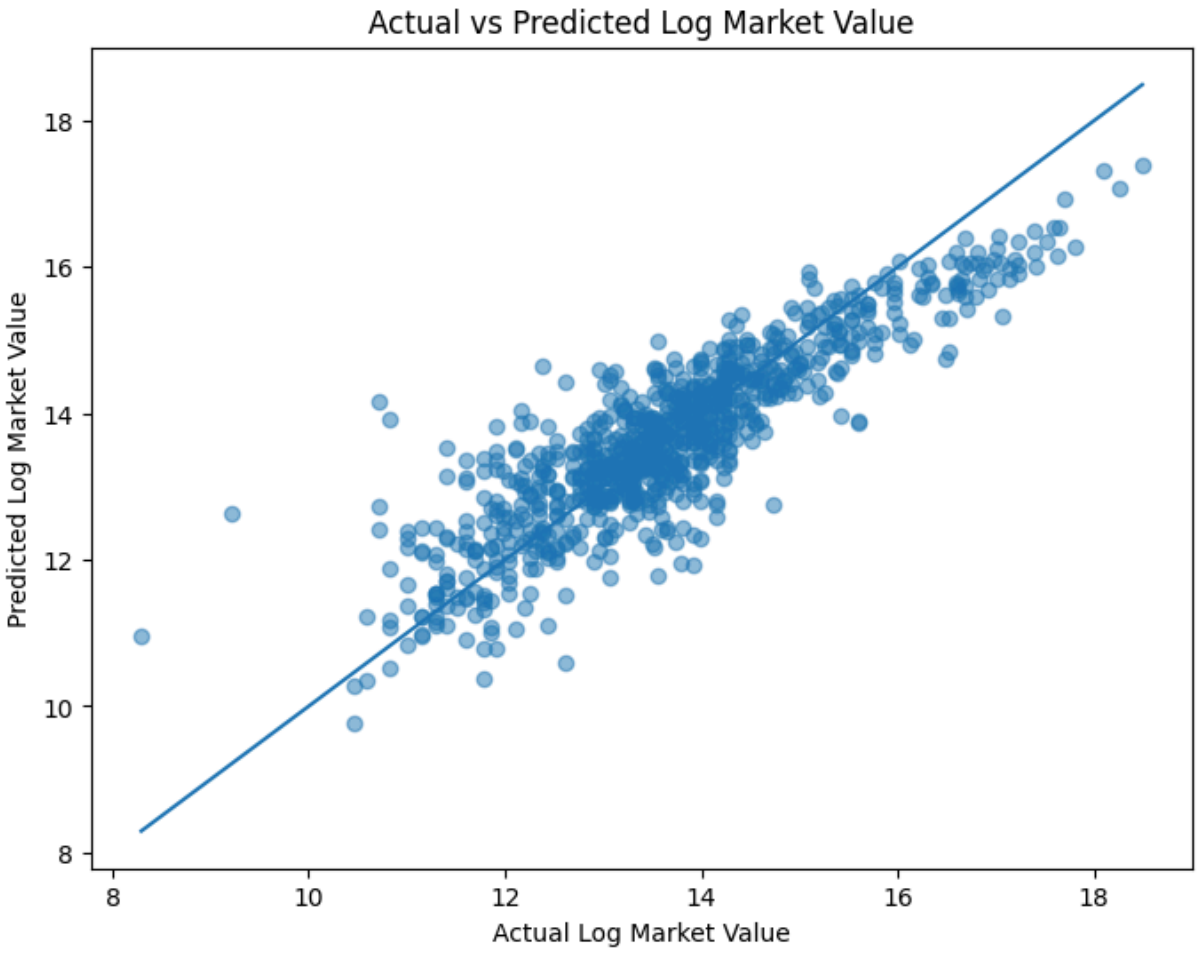
# Evaluation Metrics

In [28]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_scor
import numpy as np
```

```
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R² Score:", r2)
```

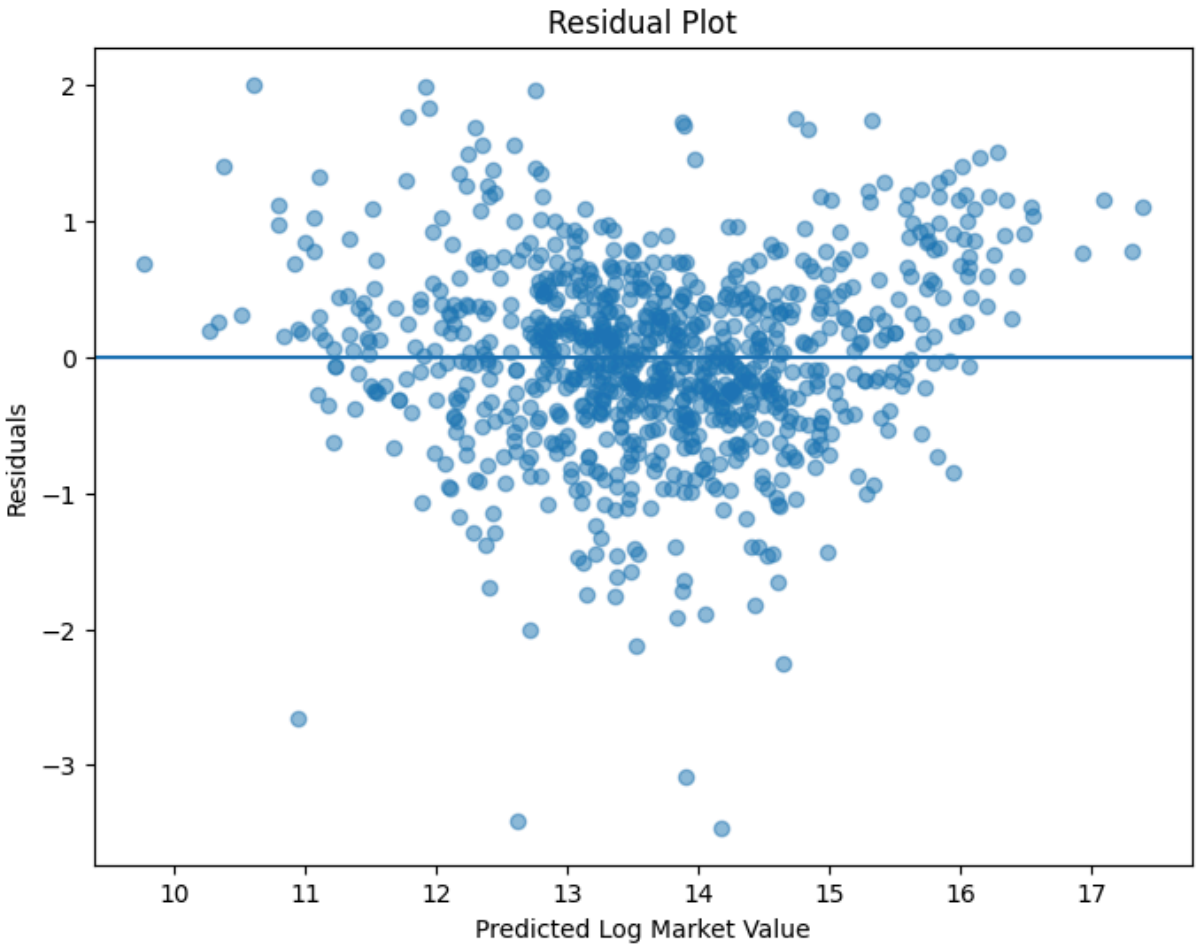
```
MAE: 0.5138587783577544
RMSE: 0.6835686253421989
R² Score: 0.7643396278587938
```

In [29]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.xlabel("Actual Log Market Value")
plt.ylabel("Predicted Log Market Value")
plt.title("Actual vs Predicted Log Market Value")

plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()])

plt.show()
```




In [30]:
```
residuals = y_test - y_pred
```

```
plt.figure(figsize=(8, 6))
plt.scatter(y_pred, residuals, alpha=0.5)
plt.axhline(y=0)
plt.xlabel("Predicted Log Market Value")
plt.ylabel("Residuals")
plt.title("Residual Plot")

plt.show()
```

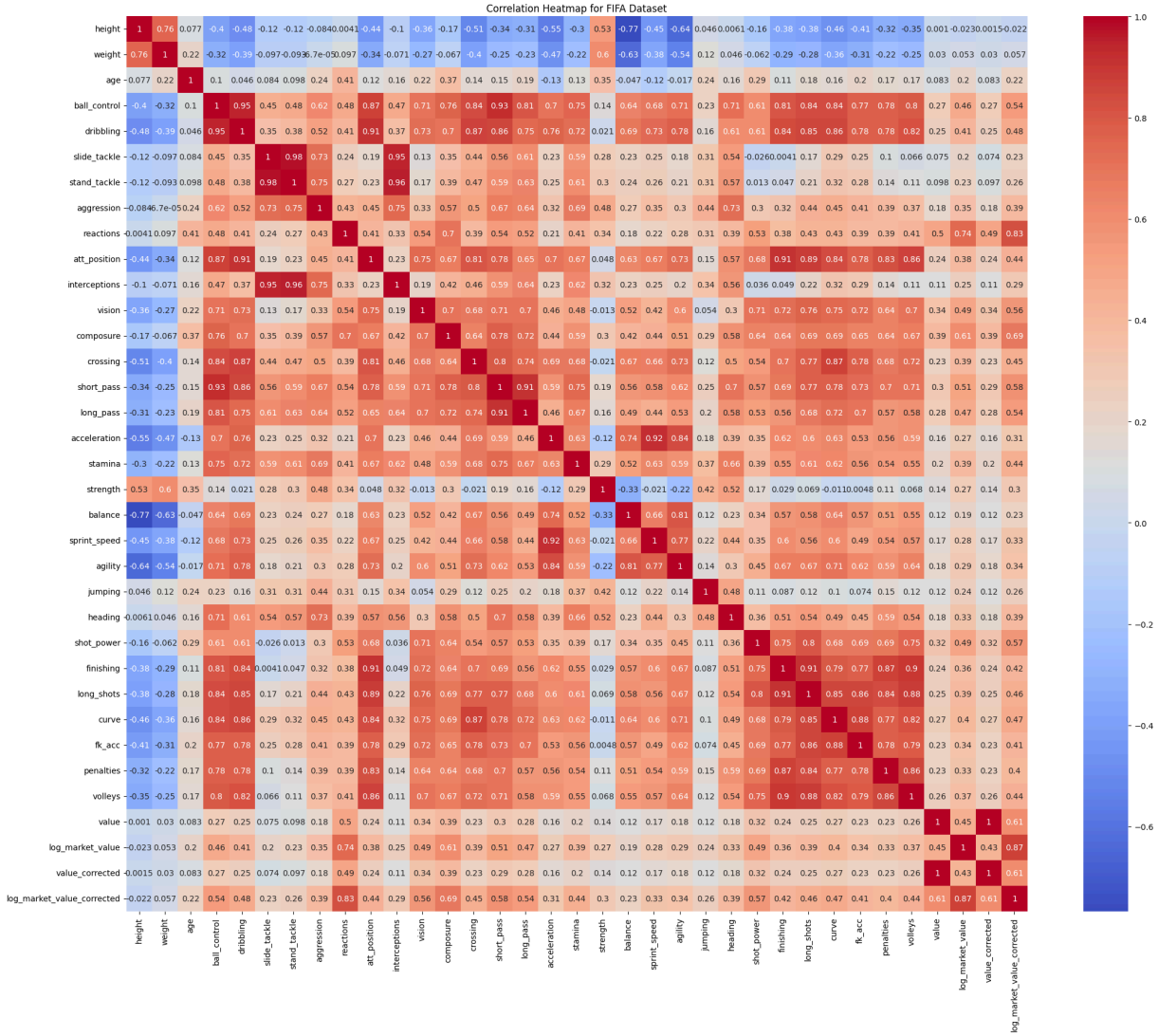



In [31]:
```
coefficients = pd.DataFrame({
    'Feature': features,
    'Coefficient': model.coef_
}).sort_values(by='Coefficient', ascending=False)

coefficients
```

Out[31]:

| | Feature | Coefficient |
|---|---|---|
| **0** | reactions | 0.101410 |
| **6** | shot_power | 0.020968 |
| **1** | composure | 0.016354 |
| **2** | short_pass | 0.011360 |
| **3** | ball_control | 0.007137 |
| **4** | vision | 0.005328 |
| **5** | long_pass | -0.009553 |
| **7** | finishing | -0.013600 |
| **8** | age | -0.046467 |

In [32]:
```
corr_matrix = df_model.select_dtypes('number').corr()

plt.figure(figsize=(25,20))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm')
plt.title("Correlation Heatmap for FIFA Dataset")
plt.show()
```




# Fine Tuning the Model

Following the textbook, we implemented GridSearch and RandomForestRegressor to fine-tune the model and define the best model.

In [33]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV

param_grid = [
    { 'n_estimators': [3, 10, 30], 'max_features': [2, 4, 6, 8] },
    { 'bootstrap': [False], 'n_estimators': [3, 10], 'max_features': [2, 3,
]

forest_reg = RandomForestRegressor(random_state=42)

grid_search = GridSearchCV( estimator=forest_reg, param_grid=param_grid, cv=
    scoring='neg_mean_squared_error', return_train_score=True, n_jobs=-1 )

grid_search.fit(X_train, y_train)
```

```
best_model = grid_search.best_estimator_

y_pred = best_model.predict(X_test)

print("Best Parameters:", grid_search.best_params_)
```

```
Best Parameters: {'max_features': 6, 'n_estimators': 30}
```

# Analyzing the Finalized Model Against Evaluation Metrics and Test Set

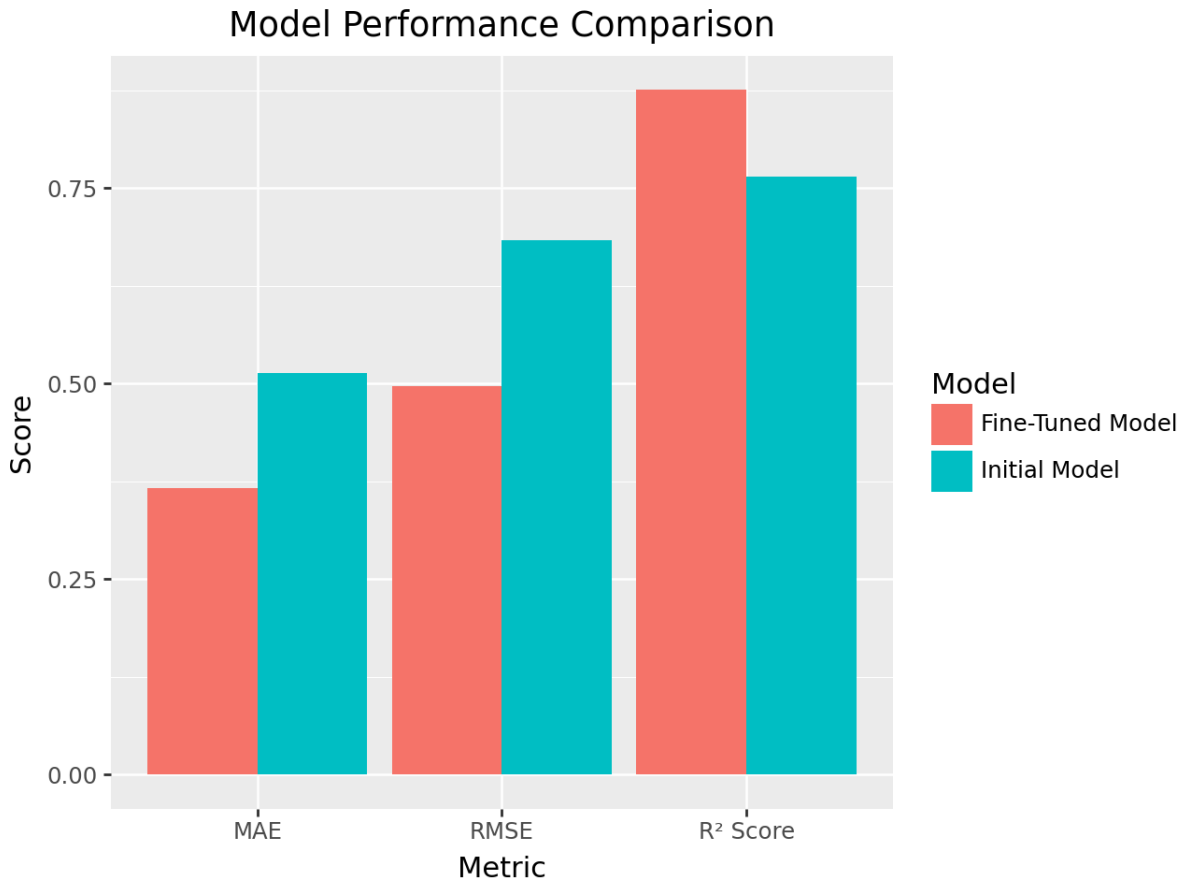
In [39]:
```
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)

print("MAE:", mae)
print("RMSE:", rmse)
print("R² Score:", r2)
```

```
MAE: 0.3666840416508452
RMSE: 0.4961778986296771
R² Score: 0.8758355846571041
```

| Metric | Initial Model | Fine-Tuned Model | Interpretation |
|---|---|---|---|
| MAE | 0.5139 | 0.5139 | Average prediction error remained about the same |
| RMSE | 0.6836 | 0.4962 | Large prediction errors were reduced significantly |
| $R^2$ Score | 0.7643 | 0.8758 | Model explains more variance in the data after tuning |

In [41]:
```
df = pd.DataFrame({
    'Metric': ['MAE', 'RMSE', 'R² Score'] * 2,
    'Score': [0.5139, 0.6836, 0.7643,
              0.3667, 0.4962, 0.8758],
    'Model': ['Initial Model'] * 3 + ['Fine-Tuned Model'] * 3
})

(
    ggplot(df, aes(x='Metric', y='Score', fill='Model'))
    + geom_bar(stat='identity', position='dodge')
    + labs(
        title='Model Performance Comparison',
        x='Metric',
        y='Score'
    )
)
```

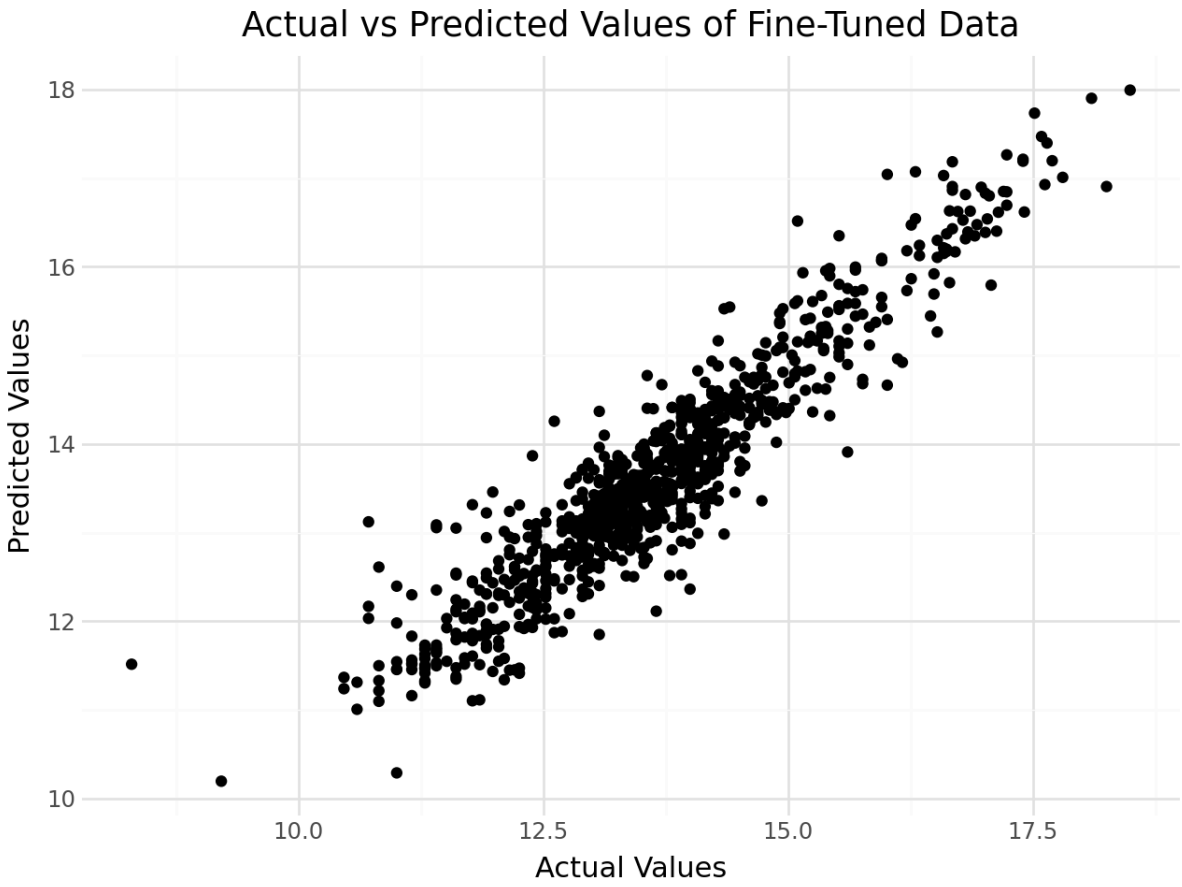
Out[41]:




In [42]:

```
scatter_df = pd.DataFrame({
    'Actual': y_test,
    'Predicted': y_pred
})

(
    ggplot(scatter_df, aes(x='Actual', y='Predicted'))
    + geom_point()
    + labs(
        title='Actual vs Predicted Values of Fine-Tuned Data',
        x='Actual Values',
        y='Predicted Values'
    )
    + theme_minimal()
)
```

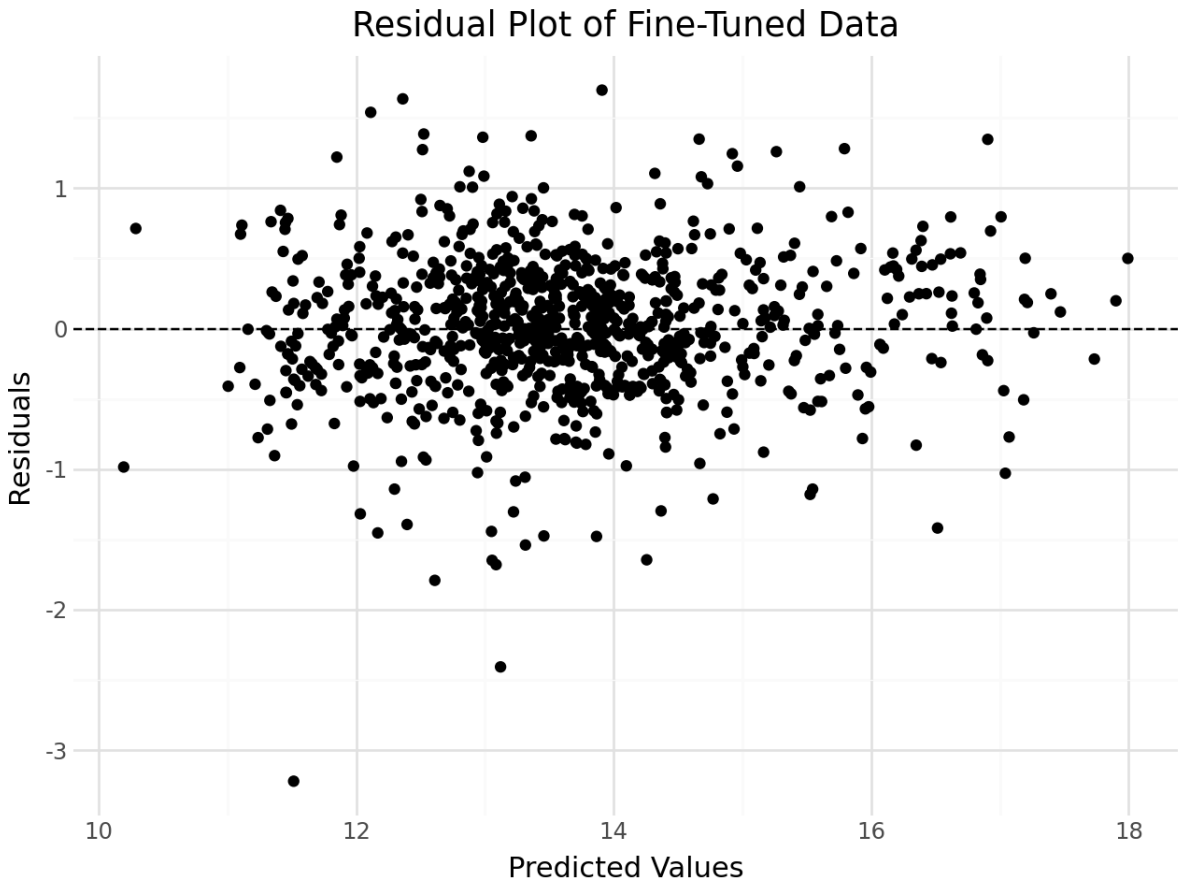
Out[42]:




In [43]:

```
residual_df = pd.DataFrame({
    'Predicted': y_pred,
    'Residuals': y_test - y_pred
})

(
    ggplot(residual_df, aes(x='Predicted', y='Residuals'))
    + geom_point()
    + geom_hline(yintercept=0, linetype='dashed')
    + labs(
        title='Residual Plot of Fine-Tuned Data',
        x='Predicted Values',
        y='Residuals'
    )
    + theme_minimal()
)
```

Out[43]:




# Saving the Model

In [38]:
```
import joblib

joblib.dump(model, "fifa_linear_regression_model.joblib")
```

Out[38]:
```
['fifa_linear_regression_model.joblib']
```

In [ ]:
```
joblib.dump(best_model, "fifa_random_forest_model.joblib")
```